

## Exercice 1. – Compter dans un arbre –

On considère un arbre *orienté* (un site distingue son père de ses fils) dans un système asynchrone. On souhaite calculer des informations relatives à cet arbre. Dans chacune des questions suivantes, on vous donne l'information que l'on souhaite calculer et vous devez écrire un algorithme distribué tel que, à la fin de l'exécution, la *racine* de l'arbre connaît l'information désirée. On suppose que les initiateurs sont les feuilles de l'arbre.

1. Calculer le nombre de *feuilles* (*i.e.*, le nombre de nœuds sans fils) contenues dans l'arbre dans le cas où il est possible de mettre des valeurs dans les messages. Indiquer le mode de terminaison (implicite ou explicite, auquel cas vous indiquerez la (ou les) action(s) STOP\_GLOBAL/STOP\_LOCAL). Donner la complexité en messages et en temps de votre algorithme, dans le pire cas.
2. Calculer le nombre de *feuilles* contenues dans l'arbre dans le cas où il est interdit de mettre des valeurs dans les messages.
  - (i) Proposer dans un premier temps, un algorithme avec terminaison implicite. Donner un exemple d'arbre dans lequel la complexité en message est  $n - 1$ , puis un autre exemple dans lequel la complexité est  $(n/2)^2$  (avec  $n$ , le nombre de nœuds dans l'arbre). En déduire la complexité en messages de votre algorithme, dans le pire cas. Quelle est sa complexité en temps dans le pire cas ?
  - (ii) Proposer dans un second temps, un algorithme avec terminaison explicite, en supposant des canaux FIFO. Puis, donner un exemple illustrant la nécessité d'avoir des canaux FIFO. Quelle est la complexité en messages et en temps de cet algorithme, dans le pire cas.
3. Calculer le nombre de nœuds *spéciaux* contenus dans l'arbre. Un nœud spécial est un nœud interne avec *exactement* un de ses fils qui est une feuille.

Connaissance :

$\text{père}_i$  : implicitement : } racine :  $\text{père}_i = \emptyset$   
 $\text{fils}_i$  : } feuille :  $\text{fils}_i = \emptyset$

2. 1) 隐式终止. 不停向上传. 只有根结点处统计数据  
 消息中不能加数字  
 Connaissance :  
 $\text{père}_i$   
 $\text{fils}_i$   
 Variable: nbfeuille.  
 Message: Msg  
 Code: Initialisé sur la feuille.  
 Si  $|\text{fils}_i| = 0$  alors  
 | Envoyer Msg à  $\text{père}_i$   
 | stop local  
 Sur la réception de Msg de  $j$   
 Si  $\text{père}_i \neq \text{null}$   
 | Envoyer Msg à  $\text{père}_i$   
 Sinon  
 nbfeuille ++  
 消息复杂度  $n-1$  (树状图)  
 最坏情况  $(n/2)^2$  (树状图)  
 时间复杂度

TD4  
 EX1.1.  
 Connaissance:  
 $\text{père}_i$   
 $\text{fils}_i$   
 Variable:  
 $\text{Nbregu}_i$ : nb. de message reçu initialisé à 0.  
 $\text{Nbfeuille}$ : compteur. initialisé à 0  
 Message: Msgcv  
 Code: (initialisé sur la feuille)  
 Si  $|\text{fils}_i| = 0$  alors  
 | Envoyer Msgcv à  $\text{père}_i$   
 | stop local  
 sur la réception de Msgcv de  $j$   
 $\text{Nbregu}_i ++$   
 $\text{Nbfeuille} \leftarrow \text{Nbfeuille} + \text{Nbregu}_i$   
 Si  $|\text{Nbregu}_i| = |\text{fils}_i|$  alors  
 | si  $\text{père}_i = \text{null}$   
 | | stop global  
 | Sinon  
 | | Envoyer Msg ( $\text{Nbfeuille}$ ) à  $\text{père}_i$   
 | | stop local

3. 显示 用 j fin()

Connaissance: fil-z  
pere-z

Variable: nb-feuille  
nb-regu

Msg: est-feuille()  
fin()

Initialisation sur les feuilles

Si fil-z == 0  
Envoyer est-feuille() à pere-z  
Envoyer fin() à pere-z  
stop local

Sur la reception est-feuille() de j

Si pere-z == null  
nb-feuille++

Si on envoie est-feuille() à pere-z.

Sur la reception de fin() de j

nb-regu++

Si nb-regu == (fil-z) et pere-z != null  
Envoyer fin() à pere  
stop local

3. 特殊结构... 子节点中有一个是叶子节点

Connaissance: fil-z  
pere-z

Variable: nb-feuille  
nbNs:  
cpt

Msg: MsgNbNs()  
est-feuille()

Initialisation sur les feuilles

Envoyer est-feuille() à pere-z  
stop local

Reception de est-feuille().

nb-feuille++  
cpt++

Si cpt == fil-z  
Si nb-feuille == 1.  
nbNs++

Si pere-z == null  
stop global

Si on envoie MsgNbNs() à pere-z  
stop local

Sur la reception de MsgNbNs() de j

cpt++  
nbNs = nbNs + K

Si cpt == fil-z

Si nb-feuille == 1.  
nbNs++

Si pere-z == null  
stop global

Si on envoie MsgNbNs() à pere-z stop local

叶子计数器++  
计数器++  
计数器 = 儿子数.  
叶子计数器 = 1  
nbNs 1.

在父节点 (根)  
- Stop  
- 儿子  
- MsgNbNs() 到父节点

计数++  
nbNs += K.  
计数器 = 儿子数  
...

**Exercice 2. – Election dans un graphe complet –**

- On rappelle qu'un graphe *complet* est un graphe dans lequel chaque site est relié à tous les autres sites. Sous l'hypothèse asynchrone, donner un algorithme d'élection dans un graphe complet et identifié. Voici les contraintes de l'algorithme :
  - On suppose que l'évènement *initialement* est généré sur un sous-ensemble non vide des sites du système.
  - Votre algorithme doit élire le site d'identifiant maximal *parmi tous les sites initiateurs*.
  - À la fin de l'exécution, chaque site doit connaître l'identifiant du leader.
  - L'algorithme peut terminer sans qu'aucun site ne le sache.
- Donner la complexité en messages et en temps de votre algorithme, dans le pire cas. Justifier brièvement.
- Donner une version de l'algorithme dans laquelle le site finalement élu est capable de détecter sa propre terminaison (STOP\_LOCAL).

1)

Comm    Comp(max<sub>i</sub>)    Diff

Locale : max<sub>i</sub>

Code : événement initial.

max<sub>i</sub> = i

Envoyer comp(max<sub>i</sub>) à {vois<sub>i</sub>}

Réception de comp(max<sub>j</sub>) de j

Si max<sub>j</sub> > max<sub>i</sub>

$\max_i \leftarrow \max_j$

complexité en message  $n^2$

temps (synchrone)  $= O(1)$

2) Local:  $\text{Card}_i \leftarrow 0$

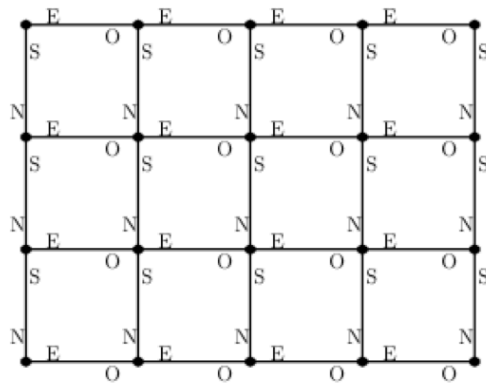
$\max_i$  nbre  
first  $\leftarrow v$

État initial: un nbre arbitraire de nœuds  $> 0$

### Exercice 3. – Problème de diffusion –

1. Dans un graphe *quelconque*, il existe un site qui possède une information ( $info_i$ ) très importante qui doit être envoyée à chacun des autres sites. Donner un algorithme pour la diffusion. Donner sa complexité en messages et en temps, dans le pire cas.
2. Si le graphe est une *grille orientée*, donner un algorithme de diffusion efficace en terme de messages envoyés. Une grille est orientée si sur chaque sommet, chaque arête incidente au sommet est étiquetée par la direction correspondante (Nord, Est, Ouest ou Sud), cf. la figure ci-dessous. Un site peut donc envoyer des messages dans les directions qu'il souhaite.

Donner la complexité en messages et en temps de votre algorithme, dans le pire cas.



Écrire 1 algo tq à la fin de l'exécution 1 noeud initiateur connaît le nombre de noeuds dans une grille orientée.

**Exercice 4.** – Sur un tore –

Un tore est un graphe de  $n = t \times t$  sommets pour lequel chaque sommet peut se voir attribuer une identité définie par le couple  $(i, j)$  avec  $0 \leq i, j < t$  de sorte que le sommet  $(i, j)$  et le sommet  $(i', j')$  sont voisins si et seulement si

- $(i = i')$  et  $j = j' + 1$ , on dira que  $(i', j')$  est en "bas" de  $(i, j)$
- $(i = i')$  et  $j = j' - 1$ , on dira que  $(i', j')$  est en "haut" de  $(i, j)$
- $(i = i' + 1)$  et  $j = j'$ , on dira que  $(i', j')$  est à "gauche" de  $(i, j)$
- $(i = i' - 1)$  et  $j = j'$ , on dira que  $(i', j')$  est à "droite" de  $(i, j)$

Attention, les additions et les soustractions sont modulo  $n$  (par exemple si  $n = 3$ ,  $1 + 2 = 0$ ,  $0 - 1 = 2$ )

1. Donner une illustration d'un tore avec  $n = 3 \times 3 = 9$  sommets.
2. Un tore de  $n = t \times t$  sommets est il un graphe régulier ? Si oui, donner le degré d'un tore.
3. Quel est le nombre d'arêtes d'un tore de  $N = t \times t$  sommets.
4. Quel est le diamètre d'un tore de  $N = t \times t$  sommets.

Dans la suite, on cherche à construire un algorithme efficace de diffusion sur un tore.

*Indication:* le tore est un graphe hamiltonien (*i.e.*, il est possible de construire un anneau sur le tore), la stratégie pour construire l'algorithme de parcours devra s'inspirer de ce résultat. Par exemple, à partir d'un site initiateur, l'idée est de rediriger le jeton uniquement vers les directions en "haut" et à "droite". Après  $k$  étapes, le jeton sera envoyé dans une direction si  $t$  est divisible par  $k$  et dans une autre sinon. L'algorithme termine lorsque le jeton revient sur l'initiateur.

5. Donner le pseudo-code de l'algorithme
6. Donner la complexité en message et la complexité en temps de l'algorithme, dans le pire cas.